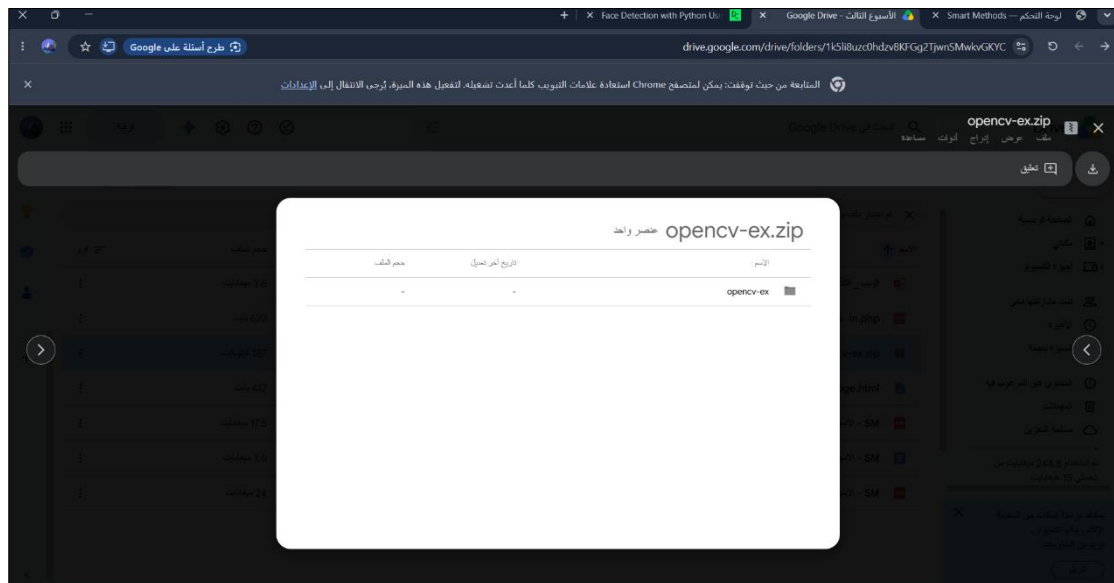


Color Detection Using OpenCV and Python

This project uses **Python** and **OpenCV** to create a real-time color detection system. The program uses a webcam to detect **red objects** and highlights them with a green rectangle.

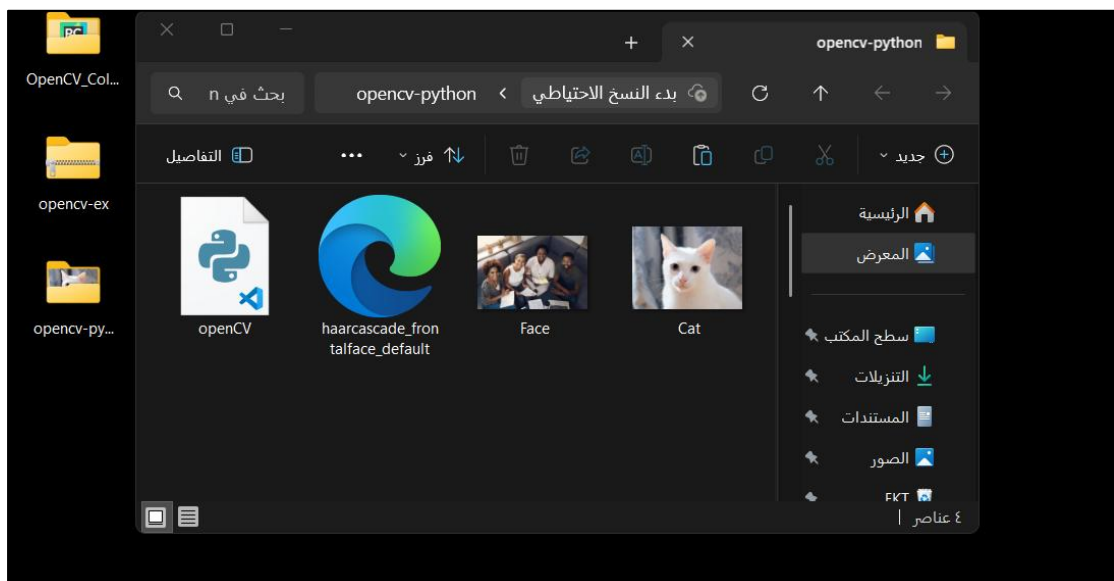
Step 1: Download OpenCV

Download the **OpenCV ZIP file** required for the project.



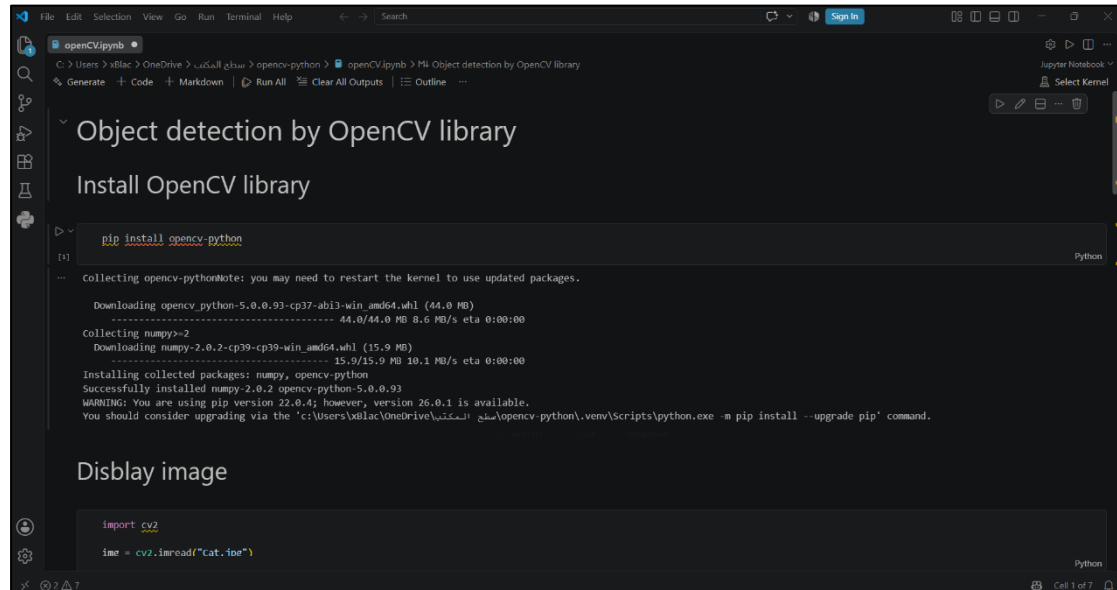
Step 2: Extract the File

After downloading the ZIP file, **extract its contents** to a suitable location on your computer.



Step 3: Open Visual Studio Code

Open **Visual Studio Code** and install the required **OpenCV** library using the terminal.



The screenshot shows the Visual Studio Code interface with a Jupyter Notebook titled 'openCV.ipynb'. The notebook contains two cells. The first cell, titled 'Object detection by OpenCV library', has a sub-header 'Install OpenCV library' and contains the command `pip install opencv-python`. The output shows the installation of `opencv-python` and `numpy`. The second cell, titled 'Display image', contains the code `import cv2` and `img = cv2.imread("Cat.jpg")`. The file explorer on the left shows the project structure with files like `Cat.jpg` and `haarcascade_frontalface_default.xml`.

```
pip install opencv-python
```

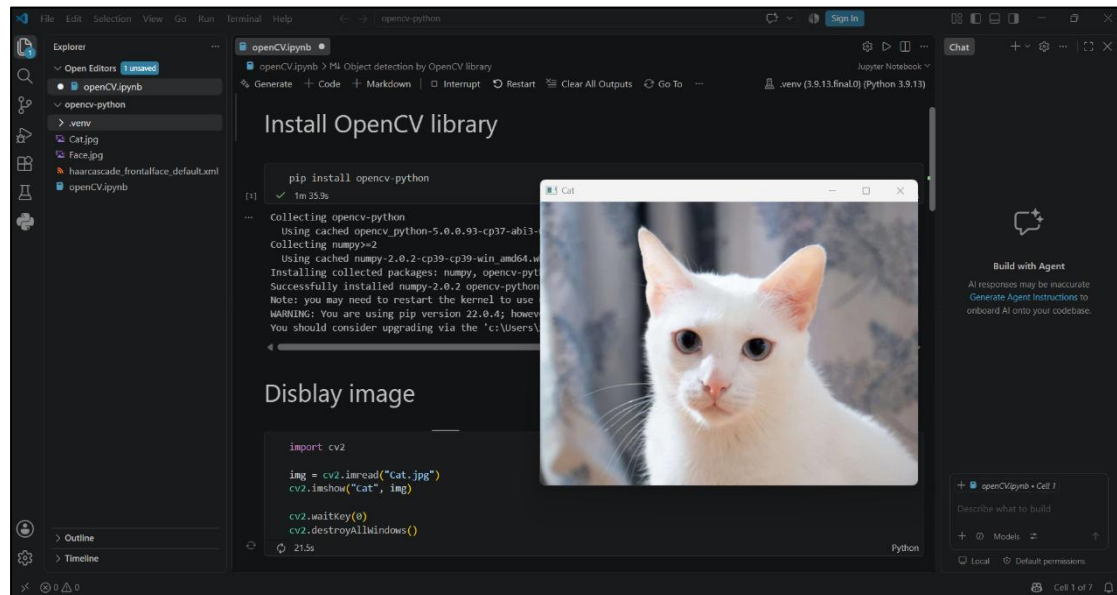
```
Collecting opencv-python
  Downloading opencv_python-5.0.0.93-cp37-abi3-win_amd64.whl (44.0 MB)
    ----- 44.0/44.0 MB 8.6 MB/s eta 0:00:00
Collecting numpy>=2
  Downloading numpy-2.0.2-cp39-cp39-win_amd64.whl (15.9 MB)
    ----- 15.9/15.9 MB 10.1 MB/s eta 0:00:00
Installing collected packages: numpy, opencv-python
Successfully installed numpy-2.0.2 opencv-python-5.0.0.93
WARNING: You are using pip version 22.0.4; however, version 26.0.1 is available.
You should consider upgrading via the 'c:\Users\xblac\OneDrive\المكتبة\opencv-python\venv\Scripts\python.exe -m pip install --upgrade pip' command.
```

```
import cv2

img = cv2.imread("Cat.jpg")
```

Step 4: Test the Installation

Run a simple test code to make sure that **OpenCV** is installed correctly and working properly.



The screenshot shows the Visual Studio Code interface with the same Jupyter Notebook. The first cell's output is now complete, showing the installation of `opencv-python` and `numpy`. The second cell, titled 'Display image', contains the code `import cv2`, `img = cv2.imread("Cat.jpg")`, `cv2.imshow("cat", img)`, `cv2.waitKey(0)`, and `cv2.destroyAllWindows()`. The output shows the execution of the code and a window titled 'Cat' displaying the image of a white cat. The file explorer on the left shows the project structure.

```
pip install opencv-python
```

```
Collecting opencv-python
  Using cached opencv_python-5.0.0.93-cp37-abi3-win_amd64.whl
Collecting numpy>=2
  Using cached numpy-2.0.2-cp39-cp39-win_amd64.whl
Installing collected packages: numpy, opencv-python
Successfully installed numpy-2.0.2 opencv-python-5.0.0.93
WARNING: You are using pip version 22.0.4; however, version 26.0.1 is available.
You should consider upgrading via the 'c:\Users\xblac\OneDrive\المكتبة\opencv-python\venv\Scripts\python.exe -m pip install --upgrade pip' command.
```

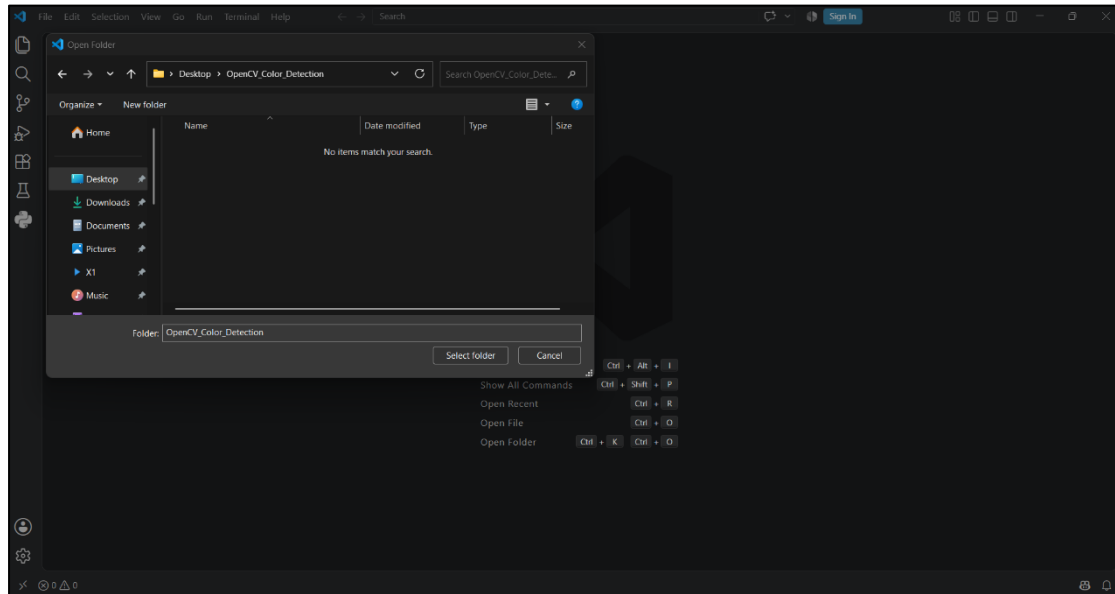
```
import cv2

img = cv2.imread("Cat.jpg")
cv2.imshow("cat", img)

cv2.waitKey(0)
cv2.destroyAllWindows()
```

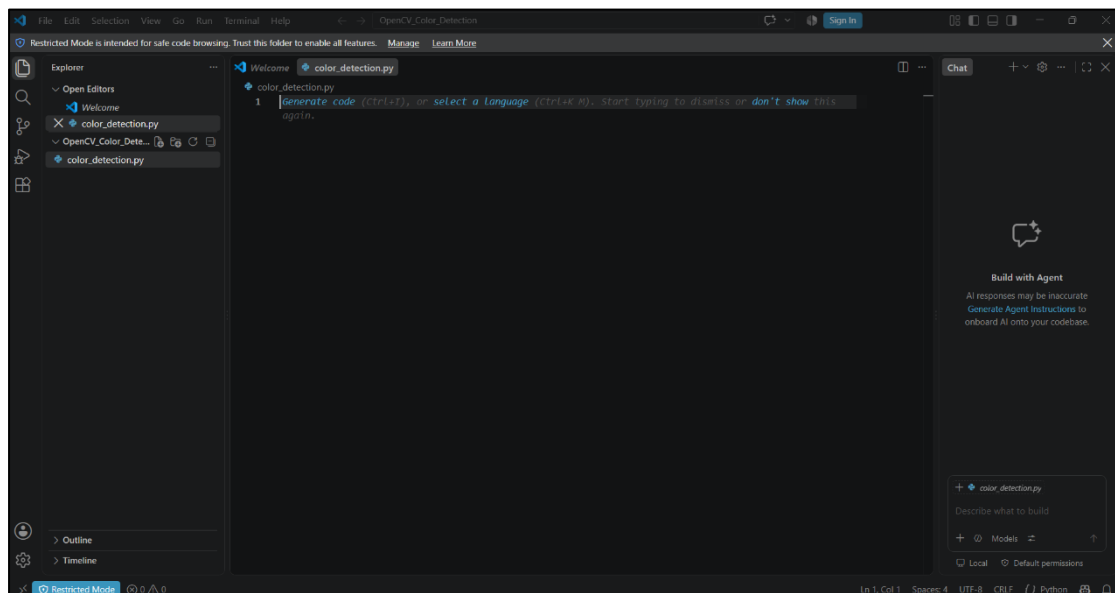
Step 5: Create the Project Folder

Create a new folder for your project and open the folder in **Visual Studio Code**.



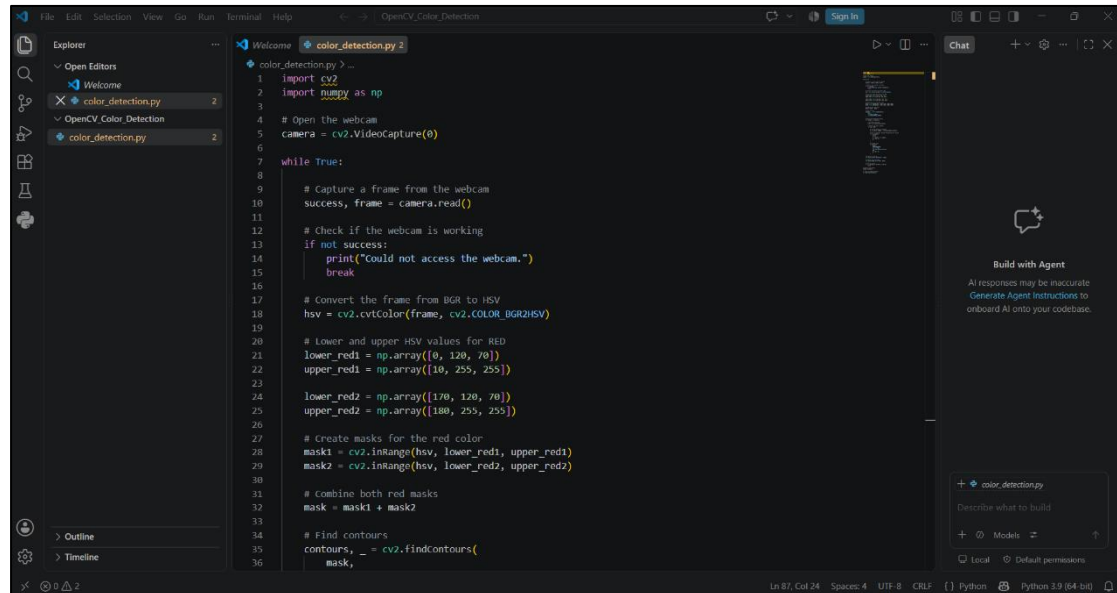
Step 6: Create a Python File

Inside the project folder, create a new Python file where you will write the program code



Step 7: Write and Run the Code

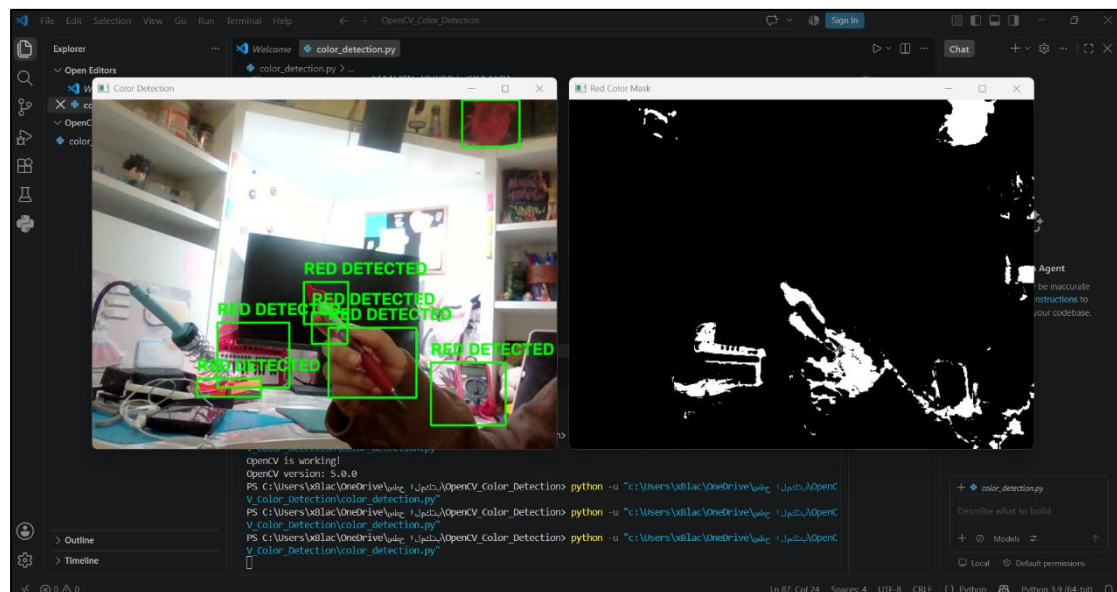
Write the color detection code in the Python file. **Make sure to save the file** before running the program. Run the code and check that the webcam opens and the program works correctly.



```
1 import cv2
2 import numpy as np
3
4 # Open the webcam
5 camera = cv2.VideoCapture(0)
6
7 while True:
8     # Capture a frame from the webcam
9     success, frame = camera.read()
10
11     # Check if the webcam is working
12     if not success:
13         print("Could not access the webcam.")
14         break
15
16     # Convert the frame from BGR to HSV
17     hsv = cv2.cvtColor(frame, cv2.COLOR_BGR2HSV)
18
19     # Lower and upper HSV values for RED
20     lower_red1 = np.array([0, 120, 70])
21     upper_red1 = np.array([10, 255, 255])
22
23     lower_red2 = np.array([170, 120, 70])
24     upper_red2 = np.array([180, 255, 255])
25
26     # Create masks for the red color
27     mask1 = cv2.inRange(hsv, lower_red1, upper_red1)
28     mask2 = cv2.inRange(hsv, lower_red2, upper_red2)
29
30     # Combine both red masks
31     mask = mask1 + mask2
32
33     # Find contours
34     contours, _ = cv2.findContours(
35         mask,
36         cv2.RETR_TREE,
37         cv2.CHAIN_APPROX_SIMPLE)
38
39     # Draw contours on the frame
40     cv2.drawContours(frame, contours, -1, (0, 255, 0), 2)
41
42     # Display the frame
43     cv2.imshow('Color Detection', frame)
44
45     # Press 'q' to quit
46     if cv2.waitKey(1) & amp; ord('q'):
```

Step 8: Results

The project successfully detects **red objects in real-time**. When a red object is placed in front of the webcam, the program displays **“RED DETECTED”** and draws a green rectangle around the detected object. A second window displays the **red color mask**, which shows the areas of the video that have been detected as red.



H.W by Eng. Raghad Hassan